fn sample_bernoulli_rational

Michael Shoemate (@Shoeboxam)

November 12, 2026

This document proves that the implementations of sample_bernoulli_rational in mod.rs at commit f5bb719 (outdated[1]) satisfies its proof definition.

1 Hoare Triple

Precondition

Compiler-Verified

- Variable prob must be of type RBig, a rational of bignums.

Caller-Verified

None

Pseudocode

```
def sample_bernoulli_rational(prob: RBig) -> bool:
    numer, denom = prob.into_parts()  #
    sign, numer = numer.into_parts()  #
    if sign == Negative:
        raise ValueError("prob must not be negative")
    if numer > denom:
        raise ValueError("prob must not be greater than one")
    return numer > sample_uniform_int_below(denom)
```

Postcondition

Theorem 1.1. For any setting of the input parameters, returns an error if there is a lack of system entropy or prob is not in $[0, 1]$, returns true with probability prob, otherwise returns false.

Proof. At 2, into_parts returns the signed numerator and unsigned nonzero denominator of prob. At 3, into_parts returns the sign and magnitude of numer.

An error is raised if the sign is negative (if prob is negative) or if the numerator is greater than the denominator (if prob is greater than one).

Since RBig.into_parts never returns a zero denominator (this would be an invalid rational), the precondition of sample_uniform_ubig_below is met, therefore by the postcondition the return value is a uniform sample in $[0, denom)$.

By countable additivity, the probability that the uniform sample is less than the numerator is exactly prob, as there are numer possible disjoint outcomes each with probability $1/$denom. It has been shown that the function returns true with probability prob, otherwise returns false. □

[1] See new changes with git diff f5bb719..138d22956 rust/src/traits/samplers/bernoulli/mod.rs